OrderServlet.java

```
package com.harisudhan.harisudhanmart.controller;

import com.harisudhan.harisudhanmart.dao.DAOFactory;
import com.harisudhan.harisudhanmart.exception.NotFoundException;
import com.harisudhan.harisudhanmart.exception.ValidationException;
import com.harisudhan.harisudhanmart.model.Order;
import com.harisudhan.harisudhanmart.service.OrderService;
import com.google.gson.JsonObject;
import java.io.IOException;
import java.sql.SQLException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

/**
 * F5 (checkout), F6 (order history for buyer + seller), O2 (status workflow).
 */
@WebServlet(urlPatterns = {
        "/api/v1/orders", "/api/v1/orders/checkout", "/api/v1/orders/mine",
        "/api/v1/orders/incoming", "/api/v1/orders/status"
})
public class OrderServlet extends BaseServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        HttpSession session = req.getSession(false);
        long userId = currentUserId(session);
        DAOFactory factory = daoFactory();
        OrderService service = new OrderService(
                factory.getDataSource(), factory.orderDAO(), factory.cartDAO(), factory.productDAO());
        String path = req.getServletPath();

        try {
            if (path.endsWith("/mine")) {
                writeJson(resp, HttpServletResponse.SC_OK, service.buyerHistory(userId));
            } else if (path.endsWith("/incoming")) {
                if (!"SELLER".equals(currentRole(session))) {
                    writeError(resp, HttpServletResponse.SC_FORBIDDEN, "FORBIDDEN", "Seller role required");
                    return;
                }
                writeJson(resp, HttpServletResponse.SC_OK, service.sellerIncomingOrders(userId));
            } else {
                writeError(resp, HttpServletResponse.SC_NOT_FOUND, "NOT_FOUND", "Unknown order endpoint");
            }
        } catch (SQLException e) {
            writeError(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "DB_ERROR", "Lookup failed");
        }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        HttpSession session = req.getSession(false);
        long userId = currentUserId(session);
        DAOFactory factory = daoFactory();
        OrderService service = new OrderService(
                factory.getDataSource(), factory.orderDAO(), factory.cartDAO(), factory.productDAO());
        String path = req.getServletPath();

        try {
            if (path.endsWith("/checkout")) {
                JsonObject body = com.google.gson.JsonParser.parseString(readBody(req)).getAsJsonObject();
```

```
                boolean confirmed = body.has("mockPaymentConfirmed") && body.get("mockPaymentConfirmed").getAsBoolean();
                writeJson(resp, HttpServletResponse.SC_CREATED, service.checkout(userId, confirmed));
            } else if (path.endsWith("/status")) {
                // Seller or admin advances an order through PENDING -> CONFIRMED -> SHIPPED -> DELIVERED (O2).
                String role = currentRole(session);
                if (!"SELLER".equals(role) && !"ADMIN".equals(role)) {
                    writeError(resp, HttpServletResponse.SC_FORBIDDEN, "FORBIDDEN", "Not permitted");
                    return;
                }
                JsonObject body = com.google.gson.JsonParser.parseString(readBody(req)).getAsJsonObject();
                long orderId = body.get("orderId").getAsLong();
                Order.Status status = Order.Status.valueOf(body.get("status").getAsString().toUpperCase());
                service.updateStatus(orderId, status);
                writeJson(resp, HttpServletResponse.SC_OK, null);
            } else {
                writeError(resp, HttpServletResponse.SC_NOT_FOUND, "NOT_FOUND", "Unknown order endpoint");
            }
        } catch (ValidationException e) {
            writeError(resp, HttpServletResponse.SC_BAD_REQUEST, e.getFieldErrorCode(), e.getMessage());
        } catch (NotFoundException e) {
            writeError(resp, HttpServletResponse.SC_NOT_FOUND, "NOT_FOUND", e.getMessage());
        } catch (SQLException e) {
            writeError(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "DB_ERROR", "Checkout failed");
        }
    }

    private String readBody(HttpServletRequest req) throws IOException {
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = req.getReader().readLine()) != null) {
            sb.append(line);
        }
        return sb.length() == 0 ? "{}" : sb.toString();
    }
}
```